



MIAGE- L3

Systèmes Informatiques

Système d'exploitation

Gestion de processus

BAKAYOKO Ibrahima

Enseignant-chercheur en informatique

UFR Mathématiques & Informatique

Bakayoko.ibrahima1@ufhb.edu.ci

Open Our Mind for a free world

Les systèmes d'exploitation sont le cœur battant de tout ordinateur. Ils sont invisibles, mais omniprésents, et jouent un rôle essentiel dans la gestion des ressources matérielles et logicielles. Au cœur de la gestion des systèmes d'exploitation se trouve la gestion des processus.

Objectif général

- Fournir aux étudiants une compréhension approfondie de la gestion des processus dans les systèmes d'exploitation.

Objectifs spécifiques

- Comprendre le concept de processus;
- Comprendre les mécanismes de communication interprocessus;
- Comprendre et utiliser les sémaphores;
- Apprendre les mécanismes de planification des processus

PLAN

INTRODUCTION

I. LES APPELS SYSTÈME

II. LES PROCESSUS

III. LES COMMUNICATION INTER-PROCESSUS

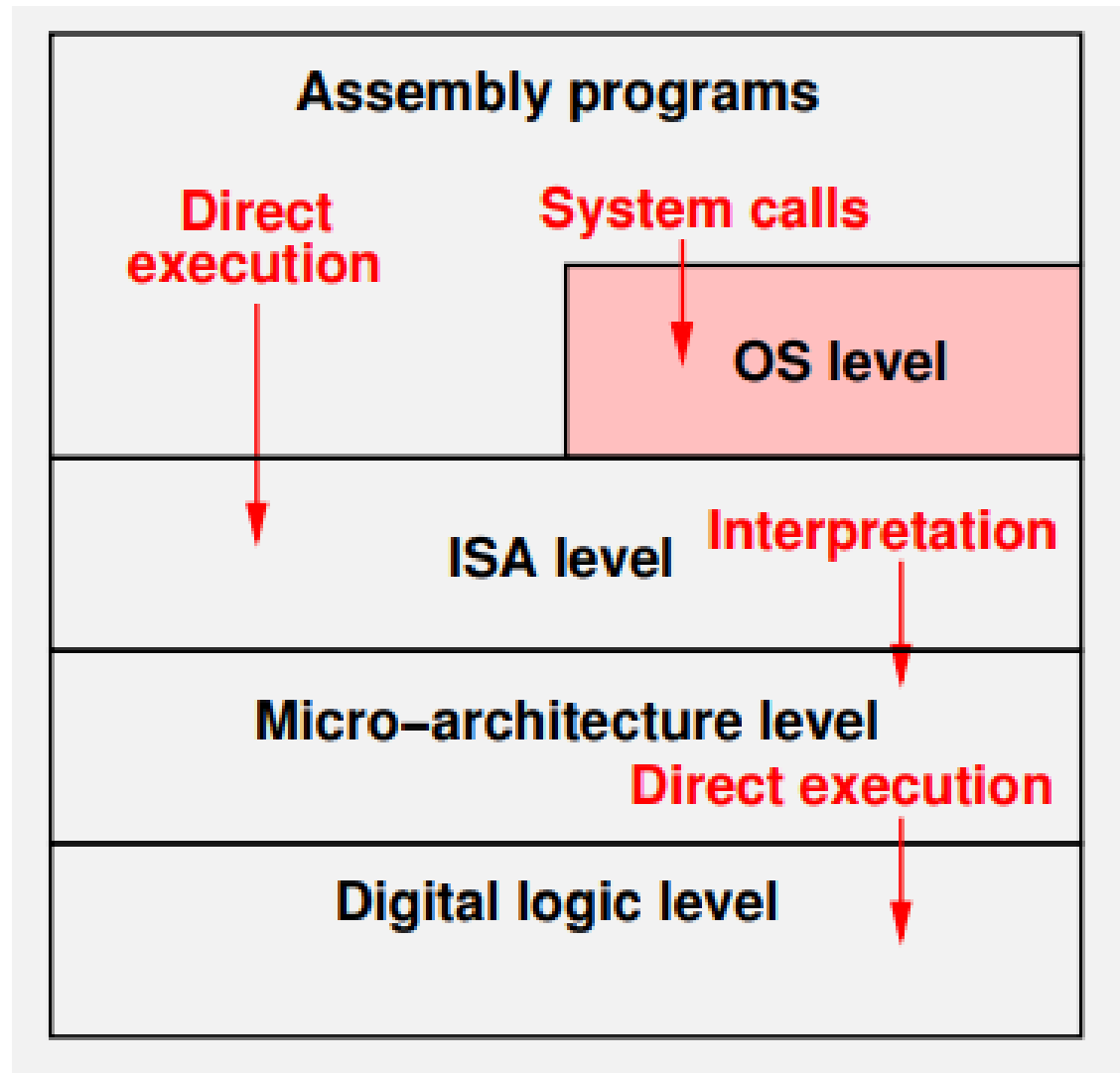
IV. LES SÉMAPHORES

V. LE PROCESSUS DE DÉMARRAGE D'UN OS

CONCLUSION

INTRODUCTION

LE NIVEAU SYSTÈME D'EXPLOITATION



QU'EST-CE QU'UN SYSTÈME D'EXPLOITATION ? 1/2

- ❑ Un système d'exploitation est une **machine étendue**
 - ❑ Il implémente des opérations de plus haut niveau que la couche ISA
 - ❑ Ces nouvelles opérations sont fournies sous forme **d'appels système**
 - ❑ Le système d'exploitation est une couche hybride : les programmes peuvent aussi utiliser la couche ISA directement

❑ Un système d'exploitation est un **gestionnaire de ressources**

- ❑ Permettre à plusieurs programmes de s'exécuter en même temps
- ❑ Permettre à plusieurs programmes de partager les ressources matérielles (mesurer l'utilisation des ressources, donner/retirer les droits d'accès aux ressources, arbitrer les conflits, etc.

- ❑ Un processus est un **programme en train de s'exécuter**
 - ❑ Espace mémoire privé (isolé des autres processus)
 - ❑ Pile privée, registres PC et SP, etc.
 - ❑ Un processus est toujours créé par un autre processus

- ❑ L'OS prend en charge **l'ordonnancement des processus**
 - ❑ A un instant donné il ne peut s'exécuter qu'un seul programme sur un CPU donné
 - ❑ L'OS fait en sorte que chaque processus reçoive une part équitable des ressources de calcul

- ❑ Les processus sont **isolés** les uns des autres
 - ❑ Chaque processus a l'illusion de pouvoir utiliser l'ensemble de la machine
 - ❑ La communication entre processus doit avoir lieu à travers des mécanismes explicites de **communication inter-processus**

- Les processus veulent pouvoir **échanger des données** avec des périphériques
- Mais chaque périphérique est différent
 - Différents types (disques, claviers, réseaux, etc.)
 - Différentes vitesses
 - Différents constructeurs
 - Etc.

- L'OS doit fournir: une interface **homogène** pour accéder à toutes les sortes de périphériques
 - Et également **arbitrer entre plusieurs processus** qui voudraient utiliser le même périphérique au même moment

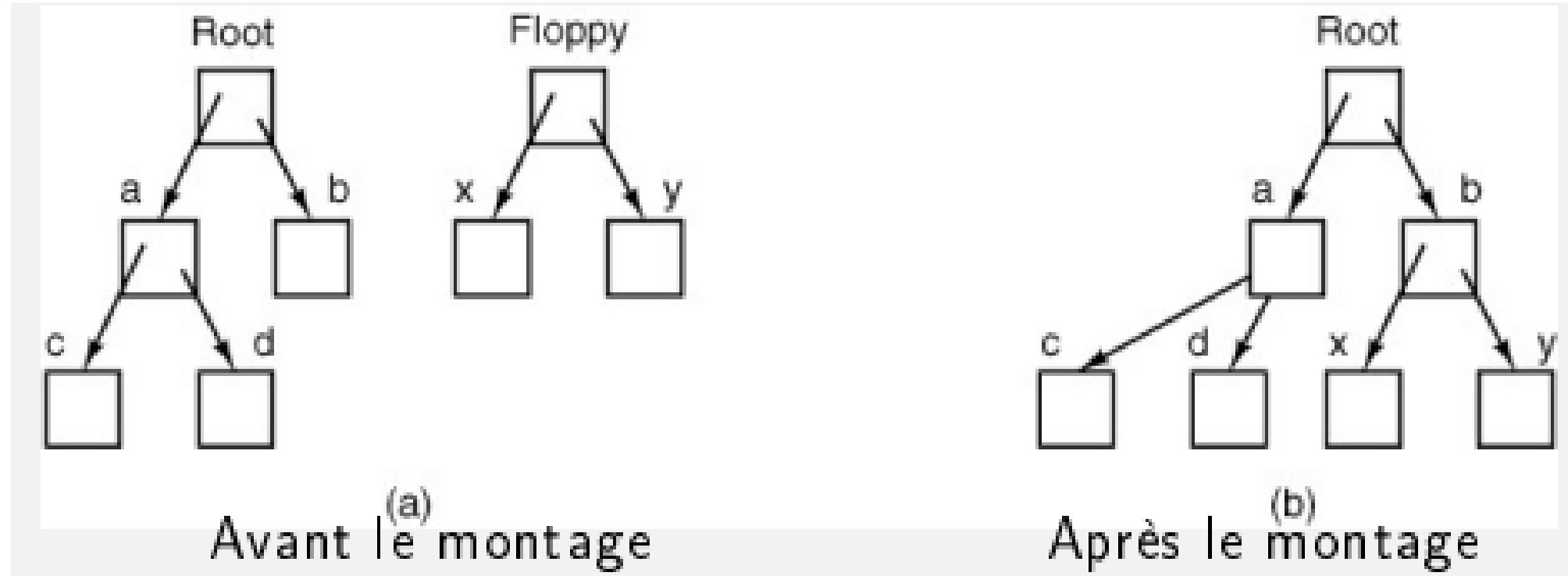
- Les processus ont besoin de mémoire pour fonctionner
 - Mais on veut isoler l'espace mémoire des différents processus
 - Mieux : donner à chaque processus l'illusion **qu'il a accès à l'intégralité de la mémoire de la machine** sans être perturbé par les autres processus

- Encore mieux : donner à chaque processus l'illusion **qu'il a accès à l'ensemble de l'espace d'adressage mémoire** (adresses $0 - 2^{32}$) indépendamment de la mémoire effectivement présente dans la machine

- L'OS doit fournir :
 - **Isolation mémoire**
 - **Pagination** (c'est-à-dire être capable de déplacer des morceaux de la mémoire vers le disque et vice-versa)
 - **Memory mapping** (c'est-à-dire donner à chaque processus accès à l'ensemble de l'espace d'adressage mémoire)

- Est-ce que quelqu'un ne sait pas ce qu'est un fichier ?
 - Comment créer des **fichiers, répertoires, permissions, etc.**
 - En utilisant des blocs de données sur disque
- On peut monter plusieurs systèmes de fichiers en une hiérarchie unique

SYSTÈMES DE FICHIERS 2/3



- Fichiers spéciaux
 - Beaucoup de concepts OS sont représentés comme des fichiers
 - Périphériques, mécanismes de communication inter-processus, etc

LES APPELS SYSTÈME

- Les appels système permettent aux processus de demander une opération au système d'exploitation

- Appels système pour la **gestion de processus** (1/2)
- `pid = fork()` Créer un processus enfant identique au parent
- `pid = waitpid(pid, &statloc, opts)` attendre la fin du process enfant
- `s = wait(&status)` ancienne version de `waitpid`
- `s = execve(name, argv, envp)` remplacer une image principale du processus
- `Exit(status)` terminer l'exécution du processus et renvoyer l'état

- Appels système pour la **gestion de processus** (2/2)
- `size = brk(addr)` définir la taille du segment de données
- `pid = getpid()` renvoie l'identifiant du processus appelant
- `pid = getpgrp()` renvoie l'id du groupe de processus de l'appelant
- `pid = setid()` Créer une nouvelle session et renvoyer son identifiant de groupe de processus
- `L = ptrace(req, pid, addr, data)` utilisé pour le débogage

- Appels système pour la **gestion de signaux** (1/2)
- `s = sigaction(sig, &act, &oldact)` définir l'action à prendre face aux signaux
- `s = sigreturn(&context)` retour d'un signal
- `s = sigprocmask(how, &set, &old)` examiner ou modifier le masque de signal
- `s = sigpending(set)` obtenir l'ensemble des signaux bloqués

- Appels système pour la **gestion de signaux** (2/2)
- `s = sigsuspend(sigmask)` remplacer le masque de signal et suspendre le processus
- `s = kill(pid, sig)` envoyer un signal à un processus
- `residual = alarm(seconds)` régler l'alarme
- `s = pause()` suspendre l'appelant jusqu'au prochain signal

- Appels système pour la **gestion de fichiers** (1/3)
- `fd = creat(name, mode)` moyen obsolète de créer un nouveau fichier
- `fd = mknod(name, mode, addr)` créer un inode régulier, spécial ou de répertoire
- `fd = open(file, how, ...)` ouvrir un fichier en lecture, en écriture ou les deux
- `s = close()` fermer un fichier ouvert
- `n = read(fd, buffer, nbytes)` lire les données d'un fichier dans un buffer

- Appels système pour la **gestion de fichiers** (2/3)
- `n = write(fd, buffer, nbytes)` écrire les données d'un buffer dans un fichier
- `pos = lseek(fd, offset, whence)` déplacer le pointeur de fichier
- `s = stat(name, &buf)` obtenir des informations sur l'état du fichier
- `s = fstat(fd, &buf)` obtenir des informations sur l'état du fichier
- `fd = dup(fd)` allouer un nouveau descripteur de fichier ouvert
- `s = pipe(&fd[0])` créer un pipe

- Appels système pour la **gestion de fichiers** (3/3)
- `s = ioctl(fd, request, argp)` effectuer des opérations spéciales sur un fichier
- `s = access(name, amode)` vérifier l'accessibilité d'un fichier
- `s = rename(old, new)` renommer un fichier
- `s = fcntl(fd, cmd, ...)` Verrouillage de fichier et autres opérations

■ Appels système pour la **gestion de répertoires et de systèmes de fichiers** (1/2)

- `s = mkdir(name, mode)` Créer un nouveau répertoire
- `s = rmdir(name)` Supprimer un répertoire vide
- `s = link(name1, name2)` Créer une nouvelle entrée, `name2` pointant vers `name1`
- `s = unlink(name)` Supprimer une entrée de répertoire
- `Mount(special, name, flag)` Monter un fichier système

■ Appels système pour la **gestion de répertoires et de systèmes de fichiers (2/2)**

- `s = unmount(special)` Démonter un fichier système
- `s = sync()` Vider tous les blocks mis en cache sur le disque
- `s = chdir(dirname)` Changer le répertoire de travail
- `s = chroot(dirname)` changer le répertoire racine

- Appels système pour la **gestion des protections**
- `s = chmod(name, mode)` modifier les droits d'un fichier
- `uid = getuid()` obtenir l'uid de l'appelant
- `s = setuid(uid)` attribuer un uid à l'appelant
- `s = setgid(gid)` attribuer un gid à l'appelant
- `s = chown(name, owner, group)` changer le propriétaire et le groupe d'un fichier
- `oldmask = umask(complmode)` changer le mode mask

- Appels système pour la **gestion du temps**
- `seconds = time(&seconds)` Obtenir le temps depuis le 1^{er} jan. 1970
- `s = stime(tp)` régler le temps
- `s = utime(file, timep)` définir l'heure du dernier accès à un fichier
- `s = times(buffer)` obtenir les temps utilisateur utilisés et système jusqu'à présent